NOSQL: CIAEC49

Course Outline

Unit I

Introduction

Introduction: Why NoSQL? The Value of Relational Databases, RDBMS vs NoSQL, CAP Theorem, Impedance Mismatch, Application and Integration Databases, Attack of the Clusters, The Emergence of NoSQL.

Unit-1
Introduction

NOSQL

- NoSQL stands for "not only SQL" and refers to a type of database that stores data in a non-tabular format. NoSQL databases are known for being easy to develop, flexible, and scalable. They are often used for applications that need to be reliable, highly available, and can handle frequent data changes.
- most agree that NoSQL databases store data in a more natural and flexible way. NoSQL, as opposed to SQL, is a database management approach, whereas SQL is just a query language, similar to the query languages of NoSQL databases.

Why NOSQL

- For almost as long as we've been in the software profession, relational databases have been the default choice for serious data storage, especially in the world of enterprise applications.
- If you're an architect starting a new project, your only choice is likely to be which relational database to use. (And often not even that, if your company has a dominant vendor.)
- There have been times when a database technology threatened to take a piece of the action, such as object databases in the 1990's, but these alternatives never got anywhere.
- After such a long period of dominance, the current excitement about NoSQL databases comes as a surprise. In this chapter we'll explore why relational databases became so dominant, and why we think the current rise of NoSQL databases isn't a flash in the pan.

Why NOSQL

- **Here are five reasons why organizations may choose to use a NoSQL database:**

1. **Scalability:** NoSQL databases are designed to scale horizontally, meaning that they can handle large amounts of data and user traffic by adding more commodity hardware. This makes it easier to handle the increasing demands of modern applications, without having to make significant changes to the underlying infrastructure.
2. **Performance**: NoSQL databases are optimized for handling large volumes of data, which means that they can deliver faster performance compared to traditional relational databases. This is particularly important for applications that require real-time data access and low latency.
3. **Flexibility:** NoSQL databases are schema-less, meaning that they can handle unstructured and semi-structured data in addition to structured data. This flexibility makes it easier to store and manage a wide variety of data types, including documents, graphs, and key-value pairs.
4. **Cost-effectiveness**: Because NoSQL databases are designed to run on commodity hardware, they are often more cost-effective than traditional relational databases, which can require expensive hardware and software licenses.
5. **Availability**: NoSQL databases are designed to handle high levels of traffic and data throughput, which means that they can provide high availability and fault tolerance. This is particularly important for mission-critical applications that require constant uptime and minimal downtime.

The Value of Relational Databases

- Relational databases have become such an embedded part of our computing culture that it's easy to take them for granted. It's therefore useful to revisit the benefits they provide.s

1.1.1. Getting at Persistent Data

- Probably the most obvious value of a database is keeping large amounts of persistent data. Most computer architectures have the notion of two areas of memory: a fast volatile "main memory" and a larger but slower "backing store." Main memory is both limited in space and loses all data when you lose power or something bad happens to the operating system.

- Therefore, to keep data around, we write it to a backing store, commonly seen a disk (although these days that disk can be persistent memory).
- The backing store can be organized in all sorts of ways. For many productivity applications (such as word processors), it's a file in the file system of the operating system.
- For most enterprise applications, however, the backing store is a database. The database allows more flexibility than a file system in storing large amounts of data in a way that allows an application program to get at small bits of that information quickly and easily.

1.1.2. Concurrency

- Enterprise applications tend to have many people looking at the same body of data at once, possibly modifying that data. Most of the time they are working on different areas of that data, but occasionally they operate on the same bit of data. As a result,
- we have to worry about coordinating these interactions to avoid such things as EX: double booking of hotel rooms.
- Concurrency is notoriously difficult to get right, with all sorts of errors that can trap even the most careful programmers. Since enterprise applications can have lots of users and other systems all working concurrently, there's a lot of room for bad things to happen.
- Relational databases help handle this by controlling all access to their data through transactions. While this isn't a cure-all (you still have to handle a transactional error when you try to book a room that's just gone), the transactional mechanism has worked well to contain the complexity of concurrency.
- Transactions also play a role in error handling. With transactions, you can make a change, and if an error occurs during the processing of the change you can roll back the transaction to clean things up.

1.1.3. Integration

- Enterprise applications live in a rich ecosystem that requires multiple applications, written by different teams, to collaborate in order to get things done.
- This kind of inter-application collaboration is awkward because it means pushing the human organizational boundaries.
- Applications often need to use the same data and updates made through one application have to be visible to others.
- A common way to do this is shared database integration where multiple applications store their data in a single database.
- Using a single database allows all the applications to use each others' data easily, while the database's concurrency control handles multiple applications in the same way as it handles multiple users in a single application.

Diffrence Between Relational Database and NOSQL

Relational Database	NoSQL
It is used to handle data coming in low velocity.	It is used to handle data coming in high velocity.
It gives only read scalability.	It gives both read and write scalability.
It manages structured data.	It manages all type of data.
Data arrives from one or few locations.	Data arrives from many locations.
It supports complex transactions.	It supports simple transactions.
It has single point of failure.	No single point of failure.
It handles data in less volume.	It handles data in high volume.
Transactions written in one location.	Transactions written in many locations.
support ACID properties compliance	doesn't support ACID properties
Its difficult to make changes in database once it is defined	Enables easy and frequent changes to database
schema is mandatory to store the data	schema design is not required
Deployed in vertical fashion.	Deployed in Horizontal fashion.

CAP THEORM

- The CAP theorem is used to makes system designers aware of the trade-offs while designing networked shared-data systems. CAP theorem has influenced the design of many distributed data systems. It is very important to understand the CAP theorem as It makes the basics of choosing any NoSQL database based on the requirements.
- ***CAP theorem*** *states that in networked shared-data systems or distributed systems, we can only achieve at most two out of three guarantees for a database:* ***C****onsistency,* ***A****vailability and* ***P****artition Tolerance.*
- *A* ***distributed system*** *is a network that stores data on more than one node (physical or virtual machines) at the same time.*
- Let's first understand C, A, and P in simple words:
- **Consistency:** means that all clients see the same data at the same time, no matter which node they connect to in a distributed system. To achieve consistency, whenever data is written to one node, it must be instantly forwarded or replicated to all the other nodes in the system before the write is deemed successful.

CAP THEORM

- **Availability:** means that every non-failing node returns a response for all read and write requests in a reasonable amount of time, even if one or more nodes are down. Another way to state this — all working nodes in the distributed system return a valid response for any request, without failing or exception.
- **Partition Tolerance:** means that the system continues to operate despite arbitrary message loss or failure of part of the system. In other words, even if there is a network outage in the data center and some of the computers are unreachable, still the system continues to perform. Distributed systems guaranteeing partition tolerance can gracefully recover from partitions once the partition heals.

CAP THEORM

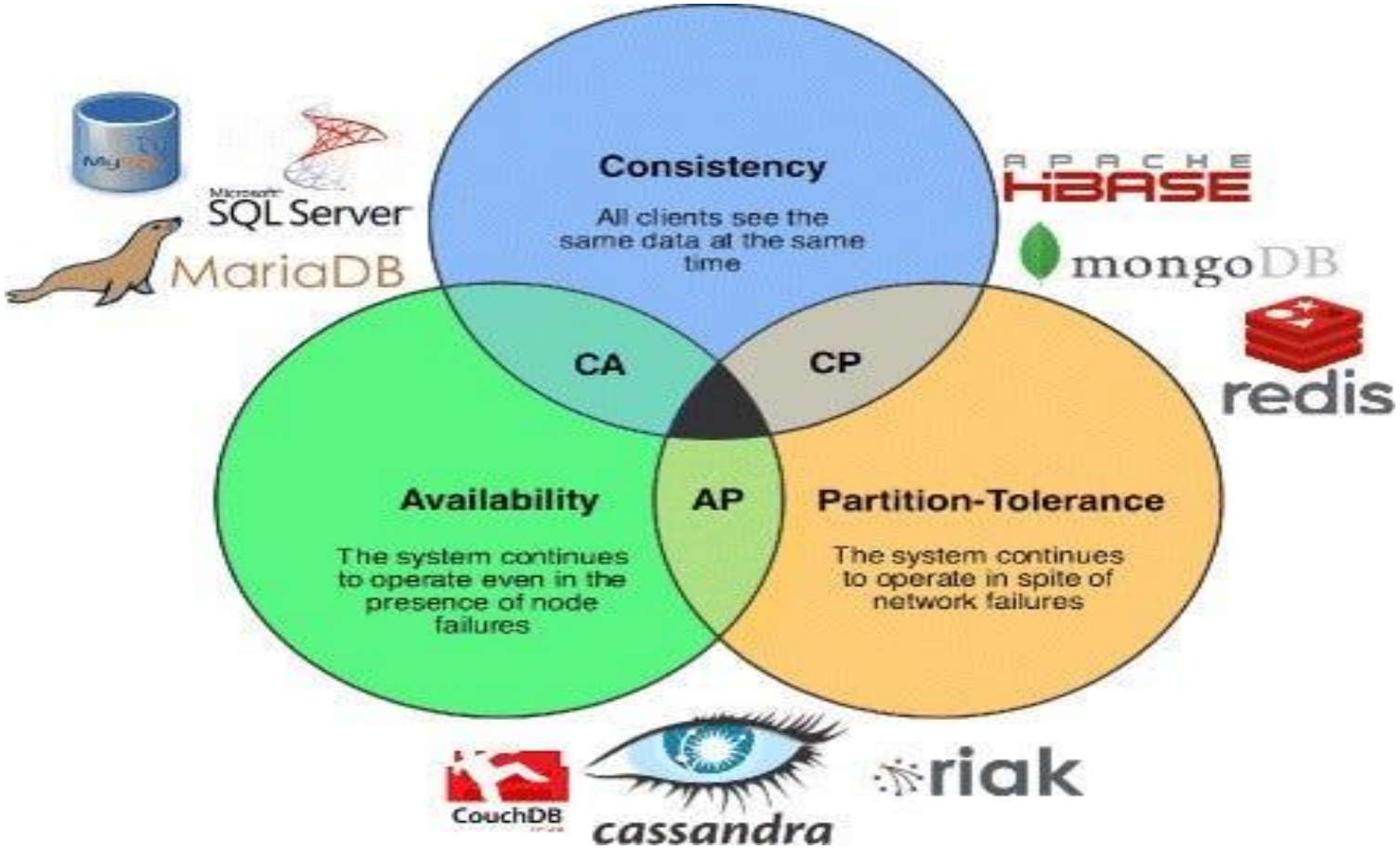
- The CAP theorem categorizes systems into three categories:
- **CP (Consistent and Partition Tolerant) database:** A CP database delivers consistency and partition tolerance at the expense of availability. When a partition occurs between any two nodes, the system has to shut down the non-consistent node (i.e., make it unavailable) until the partition is resolved.
- ***Partition*** *refers to a communication break between nodes within a distributed system. Meaning, if a node cannot receive any messages from another node in the system, there is a partition between the two nodes. Partition could have been because of network failure, server crash, or any other reason.*

CAP THEORM

- **AP (Available and Partition Tolerant) database:** An AP database delivers availability and partition tolerance at the expense of consistency. When a partition occurs, all nodes remain available but those at the wrong end of a partition might return an older version of data than others. When the partition is resolved, the AP databases typically resync the nodes to repair all inconsistencies in the system.
- **CA (Consistent and Available) database:** A CA delivers consistency and availability in the absence of any network partition. Often a single node's DB servers are categorized as CA systems. Single node DB servers do not need to deal with partition tolerance and are thus considered CA systems.
- In any networked shared-data systems or distributed systems partition tolerance is a must. Network partitions and dropped messages are a fact of life and must be handled appropriately. Consequently, system designers must choose between consistency and availability.

CAP THEORM

- The following diagram shows the classification of different databases based on the CAP theorem.



Impedence Match

- The diagram shows the classification of different databases based on the CAP theorem.
- Relational databases provide many advantages, but they are by no means perfect. Even from their early days, there have been lots of frustrations with them.
- For application developers, the biggest frustration has been what's commonly called the impedance mismatch: the difference between the relational model and the in-memory data structures.
- The relational data model organizes data into a structure of tables and rows, or more properly, relations and tuples. In the relational model, a tuple is a set of name-value pairs and a relation is a set of tuples.
- All operations in SQL consume and return relations, which leads to the mathematically elegant relational algebra.
- This foundation on relations provides a certain elegance and simplicity, but it also introduces limitations. In particular, the values in a relational tuple have to be simple— they cannot contain any structure, such as a nested record or a list. This limitation isn't true
- for in-memory data structures, which can take on much richer structures than relations. As a result, if you want to use a richer in- memory data structure, you have to translate it to a relational representation to store it on disk. Hence the impedance mismatch—two different representations that require translation

Application and Integration Databases

- The exact reasons why relational databases triumphed over OO databases are still the subject of an occasional pub debate for developers of a certain age. But in our view, the primary factor was the role of SQL as an integration mechanism between applications.
- In this scenario, the database acts asan integration database—with multiple applications, usually developed by separate teams, storing their data in a common database.
- This improves communication because all the applications are operating on a consistent set of persistent data. There are downsides to shared database integration. A structure that's designed to integrate many applications ends up being more complex—indeed, often dramatically more complex—than any single application needs. Furthermore, should an application want to make changes to its data storage, it needs to coordinate with all the other applications using the database.
- Different applications have different structural and performance needs, so an index required by one application may cause a problematic hit on inserts for another. The fact that each application is usually a separate team also means that the database usually cannot trust applications to update the data in a way that preserves database integrity and thus needs to take responsibility for that within the database itself. A different approach is to treat your database as an application database—which is only directly accessed by a single application codebase that's looked after by a single team.

Application and Integration Databases

- With an application database, only the team using the application needs to know about the database structure, which makes it much easier to maintain and evolve the schema. Since the application team controls both the database and the application code, the responsibility
- for database integrity can be put in the application code. Interoperability concerns can now shift to the interfaces of the application, allowing for better interaction protocols and providing support for changing them. During the 2000s we saw a distinct shift to web services [Daigneau], where applications would communicate over HTTP. Web services enabled a new form of a widely used communication mechanism—a challenger to using the SQL with shared databases
- An interesting aspect of this shift to web services as an integration mechanism was that it resulted in more flexibility for the structure of the data that was being exchanged. If you communicate with SQL, the data must be structured as relations.
- However, with a service,you are able to use richer data structures with nested records and lists. These are usually represented as documents in XML or, more recently, JSON. In general, with remote communication you want to reduce the number of round trips involved in the interaction,so it's useful to be able to put a rich structure of information into a single request or response.
- If you are going to use services for integration, most of the time web services—using text over HTTP—is the way to go. However, if you are dealing with highly performance-sensitive interactions, you may need a binary protocol. Only do this if you are sure you have the need, as text protocols are easier to work with—consider the example of the Internet.

Application and Integration Databases

- Once you have made the decision to use an application database, you get more freedom of choosing a database. Since there is a decoupling between your internal database and the services with which you talk to the outside world, the outside world doesn't have to care
- how you store your data, allowing you to consider nonrelational options. Furthermore, there are many features of relational databases, such as security, that are less useful to an application database because they can be done by the enclosing application instead. Despite this freedom, however, it wasn't apparent that application databases led to a big rush to alternative data stores.
- Most teams that embraced the application database approach stuck with relational databases. After all, using an application database yields many advantages even ignoring the database flexibility (which is why we generally recommend it). Relational databases are familiar and usually work very well or, at least,well enough. Perhaps, given time, we might have seen the shift to application databases to open a real crack in the relational hegemony—but such cracks came from another source.

1.4. Attack of the Clusters

- At the beginning of the new millennium the technology world was hit by the busting of the 1990s dot- com bubble. While this saw many people questioning the economic future of the Internet, the 2000s did see several large web properties dramatically increase in scale.
- This increase in scale was happening along many dimensions. Websites started tracking activity and structure in a very detailed way. Large sets of data appeared: links, social networks, activity in logs, mapping data. With this growth in data came a growth in users—as the biggest websites grew to be vast estates regularly serving huge numbers of visitors.

1.4. Attack of the Clusters

- Coping with the increase in data and traffic required more computing resources. To handle this kind of increase, you have two choices: up or out. Scaling up implies bigger machines, more processors, disk storage, and memory. But bigger machines get more and more expensive, not to mention that there are real limits as your size increases. The alternative is to use lots of small machines in a cluster.
- A cluster of small machines can use commodity hardware and ends up being cheaper at these kinds of scales. It can also be more resilient—while individual machine failures are common, the overall cluster can be built to keep going despite such failures, providing high reliability.
- As large properties moved towards clusters, that revealed a new problem—relational databases are not designed to be run on clusters. Clustered relational databases, such as the Oracle RAC or Microsoft SQL Server, work on the concept of a shared disk subsystem.

1.4. Attack of the Clusters

- They use a cluster-aware file system that writes to a highly available disk subsystem—but this means the cluster still has the disk subsystem as a single point of failure.
- Relational databases could also be run as separate servers for different sets of data, effectively sharding ("Sharding," p. 38) the database. While this separates the load, all the sharding has to be controlled by the application which has to keep track of which database server to talk to for each bit of data. Also, we lose any querying, referential integrity, transactions, or consistency controls that cross shards. A phrase we often hear in this context from people who've done this is "unnatural acts."
- These technical issues are exacerbated by licensing costs. Commercial relational databases are usually priced on a single-server assumption, so running on a cluster raised prices and led to frustrating negotiations with purchasing departments.
- This mismatch between relational databases and clusters led some organization to consider an alternative route to data storage. Two companies in particular—Google and Amazon— have been very influential. Both were on the forefront of running large clusters of this kind; furthermore, they were capturing huge amounts of data .These things gave them the motive.

1.4. Attack of the Clusters

- Both were successful and growing companies with strong technical components, which gave them the means and opportunity. It was no wonder they had murder in mind for their relational databases. As the 2000s drew on, both companies produced brief but highly influential papers about their efforts: BigTable from Google and Dynamo from Amazon.
- It's often said that Amazon and Google operate at scales far removed from most organizations, so the solutions they needed may not be relevant to an average organization.
- While it's true that most software projects don't need that level of scale, it's also true that more and more organizations are beginning to explore what they can do by capturing and processing more data—and to run into the same problems.
- So, as more information leaked out about what Google and Amazon had done, people began to explore making databases along similar lines—explicitly designed to live in a world of clusters. While the earlier menaces to relational dominance turned out to be phantoms, the threat from clusters was serious.

. The Emergence of NoSQL

- The term “NoSQL” first made its appearance in the late 90s as the name of an open source relational database Strozzi NoSQL]. Led by represented by a line with fields separated by tabs.
- The name comes from the fact that the database doesn’t use SQL as a query language. Instead, the database is manipulated through shell scripts that can be combined into tCarlo Strozzi , this database stores its tables as ASCII files, each tuple he usual UNIX pipelines
- The usage of “NoSQL” that we recognize today traces back to a meetup on June 11, 2009 in San Francisco organized by Johan Oskarsson, a software developer based in London. The example of BigTable and Dynamo had inspired a bunch of projects experimenting with alternative data storage, and discussions of these had become a feature of the better

. The Emergence of NoSQL

- software conferences around that time. Johan was interested in finding out more about some of these new databases while he was in San Francisco for a Hadoop summit. Since he had little time there, he felt that it wouldn't be feasible to visit them all, so he decided to host a meetup where they could all come together and present their work to whoever was interested.
- Johan wanted a name for the meetup—something that would make a good Twitter hashtag: short, memorable, and without too many Google hits so that a search on the name would quickly find the meetup. He asked for suggestions on the #cassandra IRC channel and got a few, selecting the suggestion of "NoSQL" from Eric Evans (a developer at Rackspace, no connection to the DDD Eric Evans).
- While it had the disadvantage of being negative and not really describing these systems, it did fit the hashtag criteria. At the time they were thinking of only naming a single meeting and were not expecting it to catch on to name this entire technology trend [Oskarsson].


Thank
You